

✓ Here's a clean, professional, and complete `README.md` for your project:

Superheroes Database - Sortable Table

A clean, interactive single-page web application built with **vanilla HTML, CSS, and JavaScript** that displays a large dataset of superheroes with powerful features: live search, multi-column sorting, and pagination.

Project Overview

This project fetches **700+ superheroes** from a public API and presents them in a beautiful, responsive table. Users can:

- Search heroes **live** by name
- Sort by any column (with smart handling for numbers like Height, Weight, and Powerstats)
- Navigate through pages (10 / 20 / 50 / 100 / All)
- View hero icons

All features work together smoothly without any external libraries (except Font Awesome for icons).

Features

- Live search (filters as you type)
- Click any column header to sort (ascending ↔ descending)
- Smart numerical sorting (Height in cm, Weight in kg, Powerstats 0-100)
- Missing values are pushed to the bottom during sorting
- Fully responsive design
- Two pagination controls (top + bottom)

- Clean dark modern UI
-

File Structure

```
text
/  
├─ index.html      # Page structure  
├─ style.css       # Styling and responsiveness  
└─ script.js       # All the JavaScript logic
```

How the Application Works (Logic Flow)

The core of the app is the `render()` function.

Every user action (search, sort, change page size, click next/prev) eventually calls `render()`.

Data Flow:

1. **Fetch data** once when page loads → stored in `allHeroes`
2. **Filter** the data based on search
3. **Sort** the filtered data
4. **Paginate** (slice) the sorted data
5. **Render** headers + body + pagination

This central flow keeps the code organized and consistent.

JavaScript - Detailed Explanation (`script.js`)

Global State Variables

```
JavaScript
```

```
let allHeroes = [];           // Full list of heroes from API
let currentPage = 1;          // Current page number
let pageSize = 20;            // How many heroes per page
let sortColumn = 'name';      // Which column is being sorted
let sortDirection = 'asc';    // 'asc' or 'desc'
```

columns **Array**

Defines every column: its data key, display label, and whether it's sortable.

Core Helper Functions

getValue(hero, key)

Purpose: Gets the correct value to **display** in each table cell.

It handles deeply nested data (e.g. hero.biography.fullName ,
hero.appearance.height[1] , etc.).

Returns ' - ' when data is missing.

heightToCm(heightStr)

Converts height strings like "6'8" , "203 cm" , or "1.85 m" into centimeters (number) for proper sorting.

weightToKg(weightStr)

Converts weight strings like "180 lb" , "85 kg" into kilograms.

getSortValue(hero, key)

Purpose: Prepares value for sorting.

Returns an object { missing: true/false, value: any }.

- Missing or invalid values go to the bottom.
 - Converts powerstats, height, and weight to real numbers.
 - Converts text to lowercase for consistent string sorting.
-

Rendering Functions

renderHeaders()

- Clears and rebuilds all table headers (`<th>`).
- Adds ▲ or ▼ arrow on the currently sorted column.
- Adds click listeners to sortable columns.

`renderBody(heroes)`

- Clears the table body.
- Creates a new row for each hero.
- For the Icon column: creates an `<img>` element.
- For other columns: puts text content.

`renderPagination(totalResults)`

- Calculates total pages based on filtered results.
- Updates both top and bottom pagination controls.
- Disables buttons when at first or last page.

`render()` ← **The Brain of the App**

This is the most important function.

JavaScript

```
function render() {
  renderHeaders();

  // 1. Get current search term
  const searchText = ...;

  // 2. Filter
  let filtered = allHeroes.filter(...);

  // 3. Sort
  filtered.sort(...);

  // 4. Paginate
  let paginated = filtered.slice(...);

  // 5. Render UI
  renderBody(paginated);
  renderPagination(filtered.length);
}
```

User Interaction Functions

`sortBy(key)`

Called when user clicks a column header.

- Toggles direction if same column.
- Resets to first page.
- Calls `render()`.

`prevPage()` & `nextPage()`

Simple navigation.

`nextPage()` recalculates filtered length to work correctly after searching.

Event Listeners

- **Search input** → resets to page 1 and calls `render()`
- **Page size select** → resets to page 1 and calls `render()`

- Column headers (added dynamically)
-

HTML Structure (`index.html`)

- Sticky fixed header (title + controls)
 - Search bar + Page size dropdown
 - Main table with `id="heroesTable"`
 - Two pagination areas (top and bottom)
-

CSS Highlights (`style.css`)

- Dark futuristic theme
 - Sticky header with backdrop blur
 - Hover effects on rows and headers
 - Responsive: table becomes readable on mobile
 - Clean card-style table
-

Sorting Logic Explained

1. User clicks column → `sortBy()` updates global state
2. `render()` runs `filtered.sort()`
3. `getSortValue()` prepares comparable values
4. Missing values are sorted last
5. Direction multiplier (`1` or `-1`) flips order

This approach allows correct sorting for both text and numbers.

Search Logic

- Filters only on `hero.name`
 - Case insensitive
 - Happens on every keystroke (`input` event)
 - Always resets to page 1
-

Pagination Logic

- Calculates `totalPages = Math.ceil(filtered.length / pageSize)`
 - Shows current page / total pages
 - `slice()` is used to show only current page items
 - Changing page size or searching resets to page 1
-

Final Notes

This project demonstrates strong vanilla JavaScript skills:

- Working with real APIs
- Managing complex state
- Clean separation of concerns
- Smart data transformation (height/weight)
- Efficient DOM updates

Project Status: Complete & Ready for Submission

You can copy this entire content into a file named `README.md`.

Would you like a **shorter version** (for school submission) or this full detailed one is good?